Microsoft Teams Apps & Integration Governance

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Teams Apps & Integration Governance 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Microsoft Teams Apps & Integration Governance 5

What this solution is . . . 5

Why this matters . . . 5

How it works . . . 5

The five governance controls . . . 6

What you can control . . . 6

Limitations (be honest with buyers) . . . 6

Naming convention (used in every doc and script) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Microsoft Teams Apps & Integration Governance 7

2.1 Teams Administrator role (app management) . . . 8

2.2 MicrosoftTeams PowerShell module (current) . . . 8

2.3 Microsoft Graph read scopes for app + consent inventory . . . 8

2.4 A pilot user group for policy assignment . . . 8

2.5 A vetting criteria and register . . . 8

2.6 High-risk Graph permission list (review baseline) . . . 9

2.7 (Confirm) Cloud environment and app availability . . . 9

Checklist . . . 9

Global Requirements — Microsoft Teams Solutions 9

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing . . . 10

2. Administrative access (least-privilege Teams roles) . . . 10

3. Microsoft Teams PowerShell + Graph toolchain . . . 10

4. Identity, groups & lifecycle source of truth . . . 10

5. A pilot scope and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

3. Implementation Guide — Microsoft Teams Apps & Integration Governance 11

Step 1 — Read the current app posture (Assess) . . . 12

Step 2 — Review consented Graph permissions (Assess) . . . 12

Step 3 — Triage into allow / block / needs-review (Design) . . . 12

Step 4 — Set org-wide app settings and allow/block (Configure) . . . 12

Step 5 — Build the app permission and setup policies (Configure) . . . 13
Step 6 — Assign policies to the pilot group (Pilot) . . . 13
Step 7 — Validate the pilot (Validate) . . . 13
Step 8 — Expand org-wide (Expand) . . . 13
Step 9 — Rollback path (documented, always ready) . . . 14
Step 10 — Stand up the recurring review (Expand) . . . 14
5. Testing Guide — Microsoft Teams Apps & Integration Governance 14
Test 1 — Posture baseline is complete . . . 14
Test 2 — Consent report flags risky permissions . . . 15
Test 3 — Triage is complete and objective . . . 15
Test 4 — Org-wide allow/block takes effect . . . 15
Test 5 — Policy build makes no changes in dry-run . . . 15
Test 6 — Pilot assignment lands . . . 15
Test 7 — Pilot user experience is correct . . . 15
Test 8 — Custom/LOB upload is governed . . . 15
Test 9 — Rollback works . . . 15
Test 10 — Recurring run refreshes the catalog . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Microsoft Teams Apps & Integration Governance 16
A. Connection / permission errors . . . 16
B. A policy change "didn't take effect" for users . . . 17
C. A blocked app is still available (or a vetted app is missing) . . . 17
D. Org-wide toggle can't be set from PowerShell . . . 17
E. Custom/LOB app can't be uploaded (or can, unexpectedly) . . . 17
F. Consent report shows a high-risk permission you didn't expect . . . 18
G. `New-CsTeamsAppPermissionPolicyApp` / policy cmdlet not found . . . 18
H. Diagnostics quick reference . . . 18
I. When to escalate . . . 18
7. Best Practices — Microsoft Teams Apps & Integration Governance 18
8. Deliverables Summary — Microsoft Teams Apps & Integration Governance 19
Documentation . . . 19
PowerShell scripts . . . 20
What the buyer receives . . . 20
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Microsoft Teams Apps & Integration Governance

Every app, bot, tab, and connector added to Microsoft Teams becomes part of your collaboration surface, and many request access to chat, file, and directory data through Microsoft Graph. In most tenants that surface is ungoverned by default: users can install almost anything, custom line-of-business apps are sideloaded with no review, and third-party apps accumulate broad consented permissions nobody is watching. That is an unmanaged software supply chain sitting inside your most-used collaboration tool. This solution gives you a repeatable way to govern it — using only the Teams admin center, the MicrosoftTeams PowerShell module, and Microsoft Graph.

This is a complete, deploy-it-today package for governing Teams apps and integrations, written for a competent Microsoft 365 collaboration administrator with no prior app-governance experience.

What you get

- A plain-language Solution Overview of the app-governance model: permission and setup policies, org-wide allow/block and pinning, custom/LOB versus third-party apps, consent/permission review, and vetting.
- A Requirements Guide with verify / implement-if-missing for every app-governance prerequisite.
- An Implementation Guide of precise, ordered admin center, PowerShell, and Graph steps in the required order — pilot-first, with a documented rollback.
- Five PowerShell scripts for app + policy discovery, safe policy configuration (with `-WhatIf`), assignment validation, a consent/Graph-permission review report, and a pilot scope/assignment helper.
- Testing and Troubleshooting guides for a confident, safe rollout.

What this delivers

A measured baseline of every app permission/setup policy and every catalog app by distribution method, a risk-flagged review of the Graph permissions apps actually hold, a vetted allow list and org-wide block list, tightened permission and setup policies validated on a pilot group before going org-wide, governed custom/LOB uploads, and a recurring review that keeps the catalog and consent grants under control as they change.

The deployment model

Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Only trusted, useful apps run in Teams — permission and setup policies set, risky apps blocked, custom uploads governed, consent reviewed, and new apps vetted — the collaboration supply chain governed, proven on a safe pilot set before you scale.

1. Solution Overview — Microsoft Teams Apps & Integration Governance

What this solution is

Every app, bot, tab, connector, and message extension added to Microsoft Teams becomes part of your collaboration surface — and many of them request access to chat, channel, file, and directory data through Microsoft Graph. Left ungoverned, the Teams app catalog is an unmanaged software supply chain: any user can install almost anything, custom line-of-business (LOB) apps get sideloaded with no review, and third-party apps accumulate broad consented permissions nobody is watching. This solution is a repeatable way to **govern which apps run in Teams and what they can reach** — using the **Teams admin center** (`https://admin.teams.microsoft.com`), the **MicrosoftTeams PowerShell module**, and **Microsoft Graph**, with no third-party tooling.

It puts five controls in place and keeps them in place: **app permission policies** (which apps a user may install), **app setup policies** (which apps are installed and pinned for them), **org-wide allow/block** on the Manage apps page, **custom/LOB and third-party app management** (who can upload, what is published to the tenant catalog), and **admin-consent / Graph-permission review** for the permissions apps actually hold — all fed by an **app-vetting workflow** so new apps are approved before they are available.

Why this matters

- App governance is off by default in the permissive direction: in many tenants users can install most store apps and even upload custom apps until you deliberately scope it down.
- The risk isn't the app UI — it's the **permissions**. A message-extension or bot that has been granted `Files.Read.All`, `Mail.Read`, or `Chat.Read.All` can egress collaboration data far beyond the channel it lives in.
- Custom/LOB apps are sideloaded straight into Teams; without an upload policy and a vetting gate, unreviewed internal apps ship with whatever permissions their developer requested.
- One-time cleanups regress. New apps appear in the store, users request new integrations, and developers publish new LOB versions every month — the value is in the **standing policy set plus a recurring review**, not a single sweep.

How it works

1. The current posture is read: app permission policies (`Get-CsTeamsAppPermissionPolicy`), app setup policies (`Get-CsTeamsAppSetupPolicy`), the org-wide app settings, and the full app catalog with each app's **distribution method** — `store` (third-party), `organization` (tenant-published LOB), or `sideloaded` (uploaded) — read from Microsoft Graph (`/appCatalogs/teamsApps`).
2. Installed apps are inventoried per team and per user (Graph `installedApps`) so you can see what is actually deployed, not just what is allowed.
3. The consented **Graph permissions** each app holds are pulled from Entra — delegated grants (`oauth2PermissionGrants`) and application permissions (`appRoleAssignments`) on each app's service principal — and **risk-flagged** against a high-impact scope list (mail, files, directory, full-access).

4. Apps are triaged into **vetted** (approved, allowed), **blocked** (disallowed org-wide), and **needs-review** (unknown publisher or risky permissions) through the vetting workflow.

5. **App permission policies** are configured to allow only vetted apps for a given user population; **app setup policies** define what is installed and pinned; **org-wide settings** allow/block third-party and custom apps and set who can upload LOB apps.

6. Everything is validated on a **pilot** user group first, then expanded org-wide, and re-reviewed on a recurring cadence as the catalog and consent grants change.

The five governance controls

• **App permission policies** — control which apps a user is *allowed to install* (all apps, only specific allowed apps, all-except-blocked, or none), by class: Microsoft apps, third-party apps, custom apps.

• **App setup policies** — control which apps are *installed and pinned* for a user, the app bar order, and whether users may pin apps themselves.

• **Org-wide allow/block** — on the Manage apps page, allow or block an individual app tenant-wide; org-wide app settings toggle whole classes (third-party apps on/off, custom apps on/off).

• **Custom/LOB & third-party management** — who may upload custom apps, and what is published to the org (tenant) app catalog for internal distribution.

• **Consent & Graph-permission review** — the delegated and application permissions each app's service principal holds in Entra, reviewed and risk-flagged.

What you can control

• Which apps are on the vetted allow list and which are blocked org-wide.

• Whether third-party apps and custom (LOB) apps are permitted at all, and for whom.

• Who may upload/sideload custom apps (org-wide setting and per-policy).

• What is pinned into the app bar for each user population, and whether users can pin their own apps.

• The pilot population that receives the tightened policies first, before any org-wide change.

• The cadence of the recurring app + consent review.

Limitations (be honest with buyers)

• Some Manage apps actions (individual-app allow/block, org-wide app settings toggles, and the custom-app upload setting) are **admin-center-driven**; PowerShell/Graph parity for those specific toggles is partial and **version-dependent — verify the exact cmdlet/endpoint in your tenant** before scripting them. Policy creation, assignment, and pinning are fully scriptable via the MicrosoftTeams module.

• Microsoft has signalled that **app permission policies are converging with the org-wide/per-app availability model** on the Manage apps page; the exact management surface is evolving — this solution uses both and flags where behavior is transitioning, so **verify current behavior in your tenant**.

• Blocking or removing an app that a team actively uses is **high-impact** — a pinned workflow can disappear for users. Pilot first and communicate.

• **Government clouds (GCC/GCC High/DoD)** differ in available store apps, portal URLs (e.g. `*.office365.us` / `admin.teams.microsoft.us`), and third-party app availability — **verify in your tenant**; this guide uses the commercial paths as the default.

• Whether a specific consented permission is *appropriate* is a **security/compliance determination**; this solution surfaces and risk-flags the permissions and provides the method — the approval decision belongs to your security and app-governance stakeholders.

Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and admin center. The standard set:

Artifact	Name
App permission policy (production standard)	`App Permission Policy – Standard – Prod`
App permission policy (pilot)	`App Permission Policy – Pilot – Pilot`
App setup policy (production standard)	`App Setup Policy – Standard – Prod`
App setup policy (frontline)	`App Setup Policy – Frontline – Prod`
Org-wide app settings baseline	`Org-wide App Settings – Tenant – Prod`
Vetted allow list	`App Allow List – Vetted – Prod`
Org-wide block list	`App Block List – Blocked – Prod`
App vetting register / workflow	`App Vetting Register – Apps – AllStages`
Pilot policy target group	`App Governance Pilot – Users – Pilot`
Consent / permission review report	`Consent Review – Teams Apps – Report`

The standard deployment model

> *Read the current app posture and consent grants → design the allow/block + policy set → configure policies and org-wide settings → assign to a **pilot** group first → validate that vetted apps work and risky ones are blocked → expand org-wide and stand up the recurring review. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Microsoft Teams Apps & Integration Governance

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant + Teams licensing, least-privilege Teams roles, the MicrosoftTeams/Graph toolchain, identity/group source of truth, a pilot scope + change control, and security/compliance stakeholders) are in `global-requirements-teams.md` — this guide adds only the **app-governance-specific** prerequisites.

2.1 Teams Administrator role (app management)

Description: An account with the **Teams Administrator** role (Global Administrator only where genuinely required, e.g. granting admin consent to app permissions). **Why:** App permission/setup policies, org-wide app settings, and the Manage apps page are gated by the Teams Administrator role; reviewing/revoking consented Graph permissions in Entra needs a privileged identity role. **Verify:** Sign in to the Teams admin center (`https://admin.teams.microsoft.com`) → `Teams apps` → `Manage apps` and `Permission policies` / `Setup policies` load and are editable. (Builds on `global-requirements-teams.md` §2.) **Implement:** Have a Global Admin assign **Teams Administrator** under change control; reserve Global Administrator for consent-granting actions only.

2.2 MicrosoftTeams PowerShell module (current)

Description: The **MicrosoftTeams** PowerShell module installed on the admin workstation, signed in with modern auth / MFA. **Why:** The permission-policy, setup-policy, and assignment cmdlets (`*-CsTeamsAppPermissionPolicy`, `*-CsTeamsAppSetupPolicy`, `Grant-Cs*`) ship in this module; the provided scripts depend on it. **Verify:** `Connect-MicrosoftTeams` succeeds and `Get-CsTeamsAppPermissionPolicy` returns the Global policy. (Builds on `global-requirements-teams.md` §3.) **Implement:** `Install-Module MicrosoftTeams -Scope CurrentUser`; run `Connect-MicrosoftTeams`.

2.3 Microsoft Graph read scopes for app + consent inventory

Description: Delegated Graph scopes for read: `AppCatalog.Read.All` (Teams app catalog), `TeamworkAppSettings.Read.All` where used, `Directory.Read.All`, `Application.Read.All`, and `DelegatedPermissionGrant.Read.All` / `AppRoleAssignment.Read.All` to read consented permissions. **Why:** The Discovery and Consent-review scripts read the Teams app catalog, installed apps, and each app's delegated (`oauth2PermissionGrants`) and application (`appRoleAssignments`) permissions from Graph. **Verify:** Run `TEAMSAPPS-Discovery.ps1` (read-only) — it connects with the read scopes and reports if consent is missing. **Implement:** Consent at first `Connect-MgGraph`; where admin consent is required an admin approves the scopes once in Entra (`Identity` → `Applications` → `Enterprise applications`).

2.4 A pilot user group for policy assignment

Description: A named group/user set — `App Governance Pilot – Users – Pilot` — to receive the tightened app permission and setup policies before any org-wide change. **Why:** App policy changes are high-impact (a blocked app disappears from users); the house model validates on a pilot before expanding. **Verify:** The pilot group exists (Entra security group or an explicit user list) and members are known. **Implement:** Create the pilot group and populate it with a representative, tolerant user set (per `global-requirements-teams.md` §5).

2.5 A vetting criteria and register

Description: Agreed **risk criteria** for approving an app (publisher/Publisher Attestation & Microsoft 365 Certification status, requested Graph permissions, data residency, where it stores data) captured in the **App Vetting Register** — `App Vetting Register – Apps – AllStages`. **Why:** "Vetted" must be

objective; the allow/block lists and the workflow key off documented criteria, not case-by-case opinion. **Verify:** The register exists (a list/spreadsheet or an intake form) with columns for app name, publisher, certification status, requested permissions, decision, and reviewer. **Implement:** Define the criteria with your security/app-governance stakeholders (`global-requirements-teams.md` §6) and stand up the register/intake form before configuring allow lists.

2.6 High-risk Graph permission list (review baseline)

Description: A documented list of **high-impact Graph permissions** that trigger deeper review when an app requests/holds them (e.g. `Mail.Read`/`Mail.ReadWrite`, `Files.Read.All`/`Files.ReadWrite.All`, `Chat.Read.All`, `Directory.ReadWrite.All`, `Sites.FullControl.All`, `full_access_as_app`). **Why:** The consent-review report risk-flags apps against this list; without it, "risky" is undefined. **Verify:** The list is recorded and referenced by `TEAMSAPPS-ConsentReport.ps1` (it ships with a sensible default set you can extend). **Implement:** Start from the script's default high-risk set, adjust with your security stakeholders, and record it alongside the vetting criteria.

2.7 (Confirm) Cloud environment and app availability

Description: Confirmation of your cloud environment — **Commercial vs Government (GCC/GCC High/DoD)** — because available store apps, portal URLs (`admin.teams.microsoft.com` vs `admin.teams.microsoft.us`), and third-party app availability differ. **Why:** Portal paths and the set of installable third-party apps are environment-dependent; scripting against the wrong endpoint or assuming an app exists will fail. **Verify:** Confirm the tenant's cloud instance in the M365 admin center; note the correct admin portal URL for your environment. **Implement:** Record the environment and substitute the correct portal URLs; **verify plan/cloud-dependent app availability in your tenant** before building allow lists.

Checklist

- Teams Administrator role confirmed (Global Admin reserved for consent grants)
- MicrosoftTeams PowerShell module installed; `Connect-MicrosoftTeams` works
- Graph read scopes consented (catalog + directory + permission-grant read)
- `App Governance Pilot – Users – Pilot` group created and populated
- Vetting criteria agreed; `App Vetting Register – Apps – AllStages` stood up
- High-risk Graph permission baseline recorded
- Cloud environment (Commercial vs Government) confirmed; portal URLs noted

Global Requirements — Microsoft Teams Solutions

These requirements apply to **every** solution in the **Teams** category (Teams governance, meetings, external access, Teams Phone, apps, Teams Rooms, and Teams security/compliance). Verify each before starting a solution's implementation steps.

Vendor boundary (applies to every Teams solution). *The **Teams admin center** (admin.teams.microsoft.com), the **Microsoft 365 admin center**, the **MicrosoftTeams PowerShell module**, and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Where a capability, policy, or portal path is **plan- or version-dependent** (Teams Phone licensing, Calling Plans vs Operator Connect vs Direct Routing, Premium features, Commercial vs Government cloud), it is **cited or flagged "verify in your tenant"** — never invented. **Third-party devices and apps** (certified Teams Rooms hardware, SBCs, ISV apps) are integrated at the pattern level; their internals are deferred to the vendor's own docs. Automation is **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing

• **Description:** A Microsoft 365 tenant with **Microsoft Teams** enabled and the licensing the capability requires (e.g. Teams Phone + a calling plan/Operator Connect for voice, Teams Premium for premium features, the appropriate E-plan for advanced compliance).

• **Why required:** All policy, voice, app, and compliance configuration depends on the right plan.

• **How to verify:** In the M365 admin center confirm the Teams subscription and relevant add-ons.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Administrative access (least-privilege Teams roles)

• **Description:** Access with the least-privilege role the solution names — typically **Teams Administrator **(or Teams Communications/Device Administrator), with** Global Administrator** only where genuinely required.

• **Why required:** Teams policy, voice, app, and org-wide settings require the corresponding role.

• **How to verify:** Confirm the role in the M365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

3. Microsoft Teams PowerShell + Graph toolchain

• **Description:** A workstation with the **MicrosoftTeams** PowerShell module (and Graph PowerShell where used), modern-auth / MFA-capable admin sign-in.

• **Why required:** Configuration, automation, and the provided scripts run through Teams PowerShell / Graph.

• **How to verify:** `Connect-MicrosoftTeams` succeeds; `Get-Module MicrosoftTeams`.

• **How to implement if missing:** `Install-Module MicrosoftTeams`; confirm modern-auth sign-in.

4. Identity, groups & lifecycle source of truth

• **Description:** A defined identity/group source (Entra ID groups, and where used **Microsoft 365

Groups **which back teams) and the** lifecycle** stance (creation, naming, expiration/retention) the org expects.

- **Why required:** Teams are backed by M365 Groups; governance, provisioning, and access key off

groups and the lifecycle model.

- **How to verify:** Confirm which groups/OU model is authoritative and the lifecycle expectations.
- **How to implement if missing:** Define the group/lifecycle model before configuring governance.

5. A pilot scope and change control

- **Description:** A **pilot team / policy group / user set** to validate on before org-wide rollout,

and a **change-control** process for production policy/voice/compliance changes.

- **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**

model; policy and voice changes are high-impact.

- **How to verify:** Confirm a named pilot scope and an approval/change window are available.
- **How to implement if missing:** Create a pilot scope and agree change control before production changes.

6. Security, compliance & governance stakeholders

- **Description:** Engagement with the **security, compliance/records, and collaboration-governance**

stakeholders whose policies the configuration must satisfy (external access, information barriers, retention, DLP, app governance).

- **Why required:** Collaboration governance and compliance settings must align to enterprise policy.
- **How to verify:** Confirm the applicable policies and named approvers.
- **How to implement if missing:** Identify the owners and capture requirements before configuring governance.

*The deployment model across every Teams solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets in code.*

3. Implementation Guide — Microsoft Teams Apps & Integration Governance

Follow these steps **in order**. Posture and inventory come from the **Teams admin center** (`https://admin.teams.microsoft.com`), the **MicrosoftTeams PowerShell module**, and **Microsoft Graph**; every policy change is run with `-WhatIf` first. Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are reused verbatim.

__Model:__ Read the current app posture and consent grants, design the allow/block + policy set, configure it, assign to a __pilot__ group first, validate that vetted apps work and risky ones are blocked, then expand org-wide and stand up the recurring review. Pattern: __Assess__ → __Design__ → __Configure__ → __Pilot__ → __Validate__ → __Expand. Government cloud:__ substitute `admin.teams.microsoft.us` and the `*.office365.us` endpoints, and __verify app availability in your tenant__ — store-app sets differ.

Step 1 — Read the current app posture (Assess)

1. Teams admin center → `Teams apps` → `Manage apps`. Note the counts of Allowed vs Blocked apps and the org-wide app settings (are third-party apps and custom apps allowed?).

2. Review `Teams apps` → `Permission policies` and `Setup policies` — note the **Global (Org-wide default)** instances and any existing custom policies.

3. Run `TEAMSAPPS-Discovery.ps1` to capture the authoritative baseline via the MicrosoftTeams module and Graph: all permission/setup policies, the full app catalog with each app's **distribution method** (`store` / `organization` / `sideloaded`), and per-team installed apps. It flags sideloaded and organization-published (LOB) apps for review.

Step 2 — Review consented Graph permissions (Assess)

1. Run `TEAMSAPPS-ConsentReport.ps1` to pull each app's service principal and its **delegated** (`oauth2PermissionGrants`) and **application** (`appRoleAssignments`) permissions from Entra, risk-flagged against the high-risk baseline (Requirement 2.6). It writes `Consent Review – Teams Apps – Report`.

2. In the Teams admin center, open a few flagged apps → the **Permissions** tab to corroborate the requested Microsoft Graph permissions shown for the app.

3. Hand the report to your security/app-governance stakeholders — the *appropriateness* decision is theirs (see Overview limitations).

Step 3 — Triage into allow / block / needs-review (Design)

1. Using the discovery and consent outputs and the vetting criteria (Requirement 2.5), classify every in-use and requested app in `App Vetting Register – Apps – AllStages` into:

• **Vetted** → goes on `App Allow List – Vetted – Prod`.

• **Blocked** → goes on `App Block List – Blocked – Prod`.

• **Needs-review** → unknown publisher or high-risk permissions; stays blocked until the workflow clears it.

1. Decide the class-level org-wide stance: are third-party apps allowed at all? Are custom (LOB) apps allowed, and who may upload them?

Step 4 — Set org-wide app settings and allow/block (Configure)

1. Teams admin center → `Teams apps` → `Manage apps` → `Org-wide app settings`. Set the baseline `Org-wide App Settings – Tenant – Prod`:

• **Third-party apps** — on or off per your Step 3 decision.

• **Custom apps** (upload/sideload) — on only if you permit LOB uploads; otherwise off.

• **Microsoft apps** — typically allowed.

1. On the Manage apps page, **Allow** the apps on `App Allow List – Vetted – Prod` and **Block** the apps on `App Block List – Blocked – Prod`.

*These Org-wide-app-settings and per-app allow/block toggles are **admin-center-driven**; PowerShell/Graph parity for these specific toggles is partial and version-dependent — **verify the exact cmdlet/endpoint in your tenant** before scripting them. The scripts below drive the fully-scriptable **policies**; use the portal for these class/allow-block toggles unless you have confirmed cmdlet support.*

Step 5 — Build the app permission and setup policies (Configure)

1. Run `TEAMSAPPS-Configure.ps1 -WhatIf` to preview creating the policies (default is dry-run — no changes):

• `App Permission Policy – Pilot – Pilot` — allows only the vetted app list for each class (Microsoft / third-party / custom), all others blocked.

• `App Setup Policy – Standard – Prod` — pins the approved core apps in the app bar and sets whether users may pin their own apps.

• (Optional) `App Setup Policy – Frontline – Prod` — a tighter pinned set for frontline/shared-device users.

1. Review the previewed policy bodies. Re-run with `-Commit` to create them:

`TEAMSAPPS-Configure.ps1 -Commit`. Under the hood this uses `New-/Set-CsTeamsAppPermissionPolicy` and `New-/Set-CsTeamsAppSetupPolicy`.

Step 6 — Assign policies to the pilot group (Pilot)

1. Run `TEAMSAPPS-Set-ScopeTargeting.ps1 -WhatIf -PolicyGroup "App Governance Pilot – Users – Pilot"` to preview assigning `App Permission Policy – Pilot – Pilot` and `App Setup Policy – Standard – Prod` to the pilot members (default dry-run, no changes).

2. Re-run with `-Commit` to apply: it calls `Grant-CsTeamsAppPermissionPolicy` and `Grant-CsTeamsAppSetupPolicy` per pilot user. **Do not** touch the Global policy yet.

Step 7 — Validate the pilot (Validate)

1. Run `TEAMSAPPS-Validation.ps1 -PolicyGroup "App Governance Pilot – Users – Pilot"` — it confirms each pilot user is assigned the intended policies and that the policies allow only the vetted list.

2. As a pilot user, confirm in the Teams client: a **vetted** app installs, a **blocked** app is not available, and the pinned apps appear as configured.

3. Complete the **Testing Guide** and clear the pass/fail gate before expanding.

Step 8 — Expand org-wide (Expand)

1. Once the pilot passes the gate, promote the settings to the **Global** policies: run `TEAMSAPPS-Configure.ps1 -WhatIf` targeting the Global permission and setup policy, review, then `-Commit`. This makes `App Permission Policy – Standard – Prod` the effective org-wide default (or updates the Global instance in place).

2. Assign any population-specific policies (e.g. `App Setup Policy – Frontline – Prod`) to their groups with `TEAMSAPPS-Set-ScopeTargeting.ps1 -Commit`.

3. Communicate the change; keep a **rollback** ready (Step 9).

Step 9 — Rollback path (documented, always ready)

1. To revert a user population, re-assign the **Global (Org-wide default)** permission and setup policies with `Grant-CsTeamsAppPermissionPolicy -Global`-equivalent via `TEAMSAPPS-Set-ScopeTargeting.ps1` (it supports resetting a group back to Global), or re-Allow a blocked app on the Manage apps page.

2. Because the pilot proved the settings on a tolerant group first, a full-population rollback is rarely needed — but the path is: reset assignment → re-allow the specific app → re-review.

Step 10 — Stand up the recurring review (Expand)

1. Schedule the read-only chain monthly: `TEAMSAPPS-Discovery.ps1` → `TEAMSAPPS-ConsentReport.ps1` → `TEAMSAPPS-Validation.ps1`. Each run refreshes the catalog, new sideloaded/LOB apps, and new consent grants.

2. Route any new or changed app through `App Vetting Register – Apps – AllStages`; update the allow/block lists and re-run `TEAMSAPPS-Configure.ps1` when the vetted set changes — keeping the collaboration supply chain governed as the catalog evolves.

Order of operations

`Read posture → Review consent → Triage (allow/block/review) → Org-wide settings + allow/block → Build policies (-WhatIf → -Commit) → Assign to pilot → Validate → Expand to Global → Rollback ready → Recurring review`

5. Testing Guide — Microsoft Teams Apps & Integration Governance

Run after building the baseline and after each configuration stage. Every test is safe; the policy dry-run (`-WhatIf`) is the gate before any change, and the pilot is the gate before org-wide rollout.

Test 1 — Posture baseline is complete

Do: Run `TEAMSAPPS-Discovery.ps1`; compare its policy and catalog counts against the Teams admin center (`Manage apps`, `Permission policies`, `Setup policies`). **Expected:** All permission/setup policies are listed; every catalog app is captured with its distribution method (`store` / `organization` / `sideloaded`), and sideloaded/LOB apps are flagged.

Test 2 — Consent report flags risky permissions

Do: Run `TEAMSAPPS-ConsentReport.ps1`; inspect `Consent Review – Teams Apps – Report`. **Expected:** Each app's delegated and application Graph permissions are listed, and apps holding high-risk scopes (Requirement 2.6) are flagged for review.

Test 3 — Triage is complete and objective

Do: Review `App Vetting Register – Apps – AllStages` against the vetting criteria. **Expected:** Every in-use/requested app is classified vetted / blocked / needs-review, and the allow and block lists (`App Allow List – Vetted – Prod`, `App Block List – Blocked – Prod`) match the register.

Test 4 — Org-wide allow/block takes effect

Do: In the Teams admin center confirm the apps on the allow list are **Allowed** and those on the block list are **Blocked**; as a test user, search for a blocked app in the Teams client. **Expected:** Allowed apps are installable; a blocked app does not appear as available to install.

Test 5 — Policy build makes no changes in dry-run

Do: Run `TEAMSAPPS-Configure.ps1 -WhatIf`; afterward re-run `TEAMSAPPS-Discovery.ps1`. **Expected:** The dry-run prints the policy bodies it *would* create/update; no new or changed policy appears — `-WhatIf` truly changed nothing.

Test 6 — Pilot assignment lands

Do: Run `TEAMSAPPS-Set-ScopeTargeting.ps1 -Commit -PolicyGroup "App Governance Pilot – Users – Pilot"`, then `TEAMSAPPS-Validation.ps1 -PolicyGroup "App Governance Pilot – Users – Pilot"`. **Expected:** Every pilot user is assigned `App Permission Policy – Pilot – Pilot` and `App Setup Policy – Standard – Prod`; the Global policy is unchanged for everyone else.

Test 7 — Pilot user experience is correct

Do: As a pilot user in the Teams client, try to install a vetted app and a blocked app, and check the pinned app bar. **Expected:** The vetted app installs; the blocked app is unavailable; the configured apps are pinned in the set order.

Test 8 — Custom/LOB upload is governed

Do: As a non-privileged pilot user, attempt to upload a custom app; confirm the org-wide custom-apps setting and the permission policy's custom-app class. **Expected:** Upload is allowed only where you permitted it; where disallowed, the user cannot sideload a custom app.

Test 9 — Rollback works

Do: On a single test pilot user, reset assignment back to the Global policies via `TEAMSAPPS-Set-ScopeTargeting.ps1` (reset mode), then re-run `TEAMSAPPS-Validation.ps1`.

Expected: The user reverts to the Global (Org-wide default) app policies cleanly — the documented rollback path is proven before org-wide expansion.

Test 10 — Recurring run refreshes the catalog

Do: Re-run the read-only chain (`Discovery` → `ConsentReport` → `Validation`) in a later cycle.
Expected: New store/sideloaded/LOB apps and new consent grants appear for triage; previously governed apps remain correctly allowed/blocked — the review self-refreshes.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–9 pass; pilot users get only vetted apps, blocked apps are unavailable, pinning is correct, and rollback is proven — with zero genuine loss of a needed app
Org-wide pass	Tests 4–7 pass against the Global policies after expansion; `Consent Review – Teams Apps – Report` shows no un-triaged high-risk app before the review is declared operational

6. Troubleshooting Guide — Microsoft Teams Apps & Integration Governance

Use `TEAMSAPPS-Discovery.ps1` for posture, `TEAMSAPPS-ConsentReport.ps1` for consented permissions, `TEAMSAPPS-Validation.ps1` for assignment state, and `TEAMSAPPS-Configure.ps1` / `TEAMSAPPS-Set-ScopeTargeting.ps1` (`-WhatIf` first) for safe changes.

A. Connection / permission errors

Symptom	Cause	Remediation
`Connect-MicrosoftTeams` fails	MicrosoftTeams module missing/outdated, or MFA not completed	`Install-Module MicrosoftTeams -Scope CurrentUser`; complete the modern-auth/MFA prompt; confirm the Teams Administrator role (Requirement 2.1)
`Connect-MgGraph` scope denied	Read scopes not consented, or admin consent required	Re-run and accept the prompt; have an admin grant `AppCatalog.Read.All`, `Directory.Read.All`, `Application.Read.All`, `DelegatedPermissionGrant.Read.All`, `AppRoleAssignment.Read.All` once in Entra (Requirement 2.3)

Symptom	Cause	Remediation
`Grant-CsTeamsAppPermissionPolicy` access denied	Insufficient role for assignment	Confirm Teams Administrator; retry `TEAMSAPPS-Set-ScopeTargeting.ps1 -Commit`

B. A policy change "didn't take effect" for users

• Policy assignment and Teams client propagation take time. Confirm the assignment with `TEAMSAPPS-Validation.ps1`; then allow for propagation (assignment can take up to a few hours to surface in the client — **verify current timing in your tenant**) and have the user sign out/in of Teams.

• Confirm you assigned the **named** policy, not left the user on the **Global (Org-wide default)**. `Get-CsUserPolicyAssignment -Identity <upn> -PolicyType TeamsAppPermissionPolicy` shows the effective policy.

C. A blocked app is still available (or a vetted app is missing)

Symptom	Cause	Remediation
Blocked app still installable	App is Allowed org-wide on the Manage apps page, overriding intent, or the user is on a permissive policy	Block the app on `Manage apps`; confirm the user's permission policy uses `AllowedAppList` for that class, not `AllApps`
Vetted app not available	App not on the allowed list in the permission policy, or blocked org-wide	Add the app ID to `App Allow List – Vetted – Prod` and re-run `TEAMSAPPS-Configure.ps1 -Commit`; ensure it is Allowed on the Manage apps page

D. Org-wide toggle can't be set from PowerShell

• The org-wide app settings (third-party on/off, custom apps on/off) and individual-app allow/block on the Manage apps page are **admin-center-driven**; PowerShell/Graph parity is partial and version-dependent. Set these in the Teams admin center (`Teams apps` → `Manage apps` → `Org-wide app settings`) and **verify any cmdlet/endpoint in your tenant** before scripting — do not assume a cmdlet exists.

E. Custom/LOB app can't be uploaded (or can, unexpectedly)

• Custom-app upload is gated by **both** the org-wide custom-apps setting **and** the setup policy's sideloading allowance (`AllowSideLoading`). If upload is blocked when it should be allowed, enable custom apps org-wide and set `AllowSideLoading=$true` on the user's setup policy; if upload is possible when it should be blocked, turn the org-wide custom-apps setting off and set `AllowSideLoading=$false` (Requirement 2.7 / Implementation Step 4).

F. Consent report shows a high-risk permission you didn't expect

• An app holds a high-risk delegated or application permission. This is a **security/compliance decision, not an automatic action** — hand `Consent Review – Teams Apps – Report` to your stakeholders. If they decide to revoke, do it in Entra (`Enterprise applications → the app → Permissions`), not by editing Teams policy. Blocking the app in Teams stops its use but does not by itself revoke a standing Graph consent grant.

G. `New-CsTeamsAppPermissionPolicyApp` / policy cmdlet not found

• The app-object helper cmdlets and the exact permission-policy parameter set are **module-version-dependent**, and Microsoft is converging permission policies with the Manage apps availability model. Update the module (`Update-Module MicrosoftTeams`), run `Get-Command *CsTeamsAppPermissionPolicy*` to confirm the available cmdlets, and **verify the current parameter names in your tenant** — do not assume a parameter that your module version doesn't expose.

H. Diagnostics quick reference

```
Connect-MicrosoftTeams
Get-CsTeamsAppPermissionPolicy | Select Identity, GlobalCatalogAppsType, PrivateCatalogAppsType
Get-CsTeamsAppSetupPolicy | Select Identity, AllowUserPinning, AllowSideLoading
Get-CsUserPolicyAssignment -Identity user@contoso.com -PolicyType TeamsAppPermissionPolicy
Connect-MgGraph -Scopes 'AppCatalog.Read.All' -NoWelcome
(Invoke-MgGraphRequest -Method GET -Uri "/v1.0/appCatalogs/teamsApps?`$top=5").value | Select id, distribu
```

I. When to escalate

If the role is Teams Administrator, Graph scopes are consented, and `TEAMSAPPS-Validation.ps1` confirms the correct policy is assigned but the client behavior is still wrong after propagation, capture the `Get-CsUserPolicyAssignment` output, the app's ID and distribution method, and the Manage-apps allow/block state, and open a Microsoft support case. For a consented-permission concern, route through your security/app-governance stakeholders before revoking anything.

7. Best Practices — Microsoft Teams Apps & Integration Governance

• **Govern the permissions, not just the icons.** The risk in a Teams app is the Graph permissions it holds. Review `Consent Review – Teams Apps – Report` on every cycle and treat a new high-risk grant (mail, files, directory, full-access) as an event, not a footnote.

• **Default to a curated allow list.** Allowing "all apps except a block list" is a losing race against the store. Allow only vetted apps for most users; expand the list deliberately through the vetting workflow.

• **Always dry-run before you change a policy.** Run `TEAMSAPPS-Configure.ps1 -WhatIf` and `TEAMSAPPS-Set-ScopeTargeting.ps1 -WhatIf` first — `-WhatIf` is the default for a reason. Committing is a separate, deliberate step.

• **Pilot before Global, every time.** Assign tightened policies to `App Governance Pilot – Users – Pilot` first; a blocked app disappearing from a user's app bar is high-impact. Prove it on a tolerant

group, then expand.

• **Vet objectively, on documented criteria.** Approve apps against `App Vetting Register – Apps – AllStages` (publisher, Microsoft 365 Certification / Publisher Attestation, requested permissions, data residency) — not on a case-by-case hunch.

• **Control custom/LOB uploads deliberately.** Sideloaded apps ship with whatever permissions their developer requested. Permit uploads only where needed, gate LOB publishing to the tenant catalog behind the vetting workflow, and keep the org-wide custom-apps setting off for populations that don't need it.

• **Keep a rollback ready.** Reset a population to the Global (Org-wide default) policies or re-Allow a specific app on the Manage apps page. Because you piloted first, a broad rollback should be rare — but the path stays documented.

• **Separate the technical method from the risk decision.** You provide the inventory and risk flags; whether a permission is *acceptable* is your security/app-governance stakeholders' call. Don't approve risky consent unilaterally.

• **Reserve Global Admin for consent grants.** Do day-to-day app governance as Teams Administrator; use Global Administrator only to grant/revoke tenant-wide admin consent — least privilege throughout.

• **Verify plan- and cloud-dependent behavior in your tenant.** Store-app availability, portal URLs, and the exact PowerShell/Graph parity for org-wide toggles differ by cloud (Commercial vs GCC/GCC High/DoD) and drift between module versions — confirm before scripting, never assume.

• **Make it recurring, not a one-off.** A one-time cleanup regresses as the store, user requests, and LOB versions change. Schedule the read-only `Discovery` → `ConsentReport` → `Validation` chain monthly and route new apps through the register.

8. Deliverables Summary — Microsoft Teams Apps & Integration Governance

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The app-governance model: permission + setup policies, org-wide allow/block + pinning, custom/LOB vs third-party, consent/permission review, and vetting; limitations and the naming convention
2. Requirements Guide	App-governance-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Teams)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered admin center + MicrosoftTeams PowerShell + Graph steps (read posture, review consent, triage, org-wide settings, build policies, assign to pilot, validate, expand, rollback, recurring review)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`TEAMSAPPS-Discovery.ps1`	Read-only inventory of app permission/setup policies, the full Teams app catalog with each app's distribution method (`store`/`organization`/`sideloaded`), and per-team installed apps; risk-flags sideloaded/LOB apps. MicrosoftTeams + Graph
`TEAMSAPPS-Configure.ps1`	Creates/updates app permission and app setup policies from a definition (vetted allow list, pinned apps); `-WhatIf` **by default**, `-Commit` to apply. Uses `New-/Set-CsTeamsAppPermissionPolicy` and `New-/Set-CsTeamsAppSetupPolicy`
`TEAMSAPPS-Validation.ps1`	Read-only check that a user population is assigned the intended permission/setup policies and that those policies allow only the vetted set
`TEAMSAPPS-ConsentReport.ps1`	Read-only consent/permission review: each app's service principal with its delegated (`oauth2PermissionGrants`) and application (`appRoleAssignments`) Graph permissions, risk-flagged. Writes `Consent Review – Teams Apps – Report`
`TEAMSAPPS-Set-ScopeTargeting.ps1`	Scope/health helper: assigns (or resets to Global) the permission + setup policies for a pilot group/users via `Grant-Cs*`; `-WhatIf` **by default**, `-Commit` to apply

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP (read-only or `-WhatIf` by default; no credentials in code).
- A repeatable process following the model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Only trusted, useful apps run in Microsoft Teams — app permission and setup policies set, risky apps blocked org-wide, custom/LOB uploads governed, consented Graph permissions reviewed in `Consent Review – Teams Apps – Report`, and new apps vetted through `App Vetting Register – Apps – AllStages` — with the collaboration supply chain governed on a recurring review.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TEAMSAPPS-Configure.ps1

Purpose: Configures the governance policies: an app permission policy that allows only the vetted app list per class (Microsoft / third-party / custom), and an app setup policy that pins the approved core apps and sets whether users may pin/sideload. SAFE BY DEFAULT — runs in -WhatIf mode (prints the policy bodies it WOULD create/update) unless -Commit is supplied. Uses the MicrosoftTeams cmdlets New-/Set-CsTeamsAppPermissionPolicy and New-/Set-CsTeamsAppSetupPolicy. HOW TO RUN Dry-run (default, NO changes): .\TEAMSAPPS-Configure.ps1 Apply: .\TEAMSAPPS-Configure.ps1 -Commit Custom names / vetted list: .\TEAMSAPPS-Configure.ps1 -PermissionPolicyName 'App Permission Policy - Pilot - Pilot' ` -VettedAppIds @('<appId1>','<appId2>') -PinnedAppIds @('<appId1>') -Commit (PowerShell 5.1+/7+; MicrosoftTeams module; Connect-MicrosoftTeams) EXPECTED RESULTS -WhatIf: prints the permission-policy allow lists and setup-policy pinned set that WOULD be applied; nothing changes. -Commit: the named policies are created or updated in place. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Default mode makes NO changes. App-permission-policy management is transitioning to the Manage apps model — verify behavior in your tenant. > [CmdletBinding()] param([string]$PermissionPolicyName = 'App Permission Policy - Pilot - Pilot', [string]$SetupPolicyName = 'App Setup Policy - Standard - Prod', [string[]]$VettedAppIds = @(), # Teams app IDs allowed for third-party + custom classes [string[]]$PinnedAppIds = @(), # Teams app IDs pinned in the app bar (in order) [ValidateSet('AllowedAppList','BlockedAppList','AllApps','NoApps')] [string]$ThirdPartyMode = 'AllowedAppList', [bool]$AllowUserPinning = $true, [bool]$AllowSideLoading = $false, # custom-app upload; keep off unless LOB uploads are permitted [switch]$Commit # absent = -WhatIf (default, no changes))

How to run: Dry-run (default, NO changes): .\TEAMSAPPS-Configure.ps1 Apply: .\TEAMSAPPS-Configure.ps1 -Commit Custom names / vetted list: .\TEAMSAPPS-Configure.ps1 -PermissionPolicyName 'App Permission Policy - Pilot - Pilot' ` -VettedAppIds @('<appId1>','<appId2>') -PinnedAppIds @('<appId1>') -Commit (PowerShell 5.1+/7+; MicrosoftTeams module; Connect-MicrosoftTeams) EXPECTED RESULTS -WhatIf: prints the permission-policy allow lists and setup-policy pinned set that WOULD be applied; nothing changes. -Commit: the named policies are created or updated in place. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Default mode makes NO changes. App-permission-policy management is transitioning to the Manage apps model — verify behavior in your tenant. > [CmdletBinding()] param([string]$PermissionPolicyName = 'App Permission Policy - Pilot - Pilot', [string]$SetupPolicyName = 'App Setup Policy - Standard - Prod', [string[]]$VettedAppIds = @(), # Teams app IDs allowed for third-party + custom classes [string[]]$PinnedAppIds = @(), # Teams app IDs pinned in the app bar (in order) [ValidateSet('AllowedAppList','BlockedAppList','AllApps','NoApps')] [string]$ThirdPartyMode = 'AllowedAppList', [bool]$AllowUserPinning = $true, [bool]$AllowSideLoading = $false, # custom-app upload; keep off unless LOB uploads are permitted [switch]$Commit # absent = -WhatIf (default, no changes))

TEAMSAPPS-ConsentReport.ps1

Purpose: Reviews what Teams apps can actually reach: for each enterprise application (service principal), lists its DELEGATED permissions (oauth2PermissionGrants) and APPLICATION permissions

(appRoleAssignments) against Microsoft Graph, and risk-flags any that appear in the high-risk scope list (mail, files, directory, chat, full-access). Read-only — makes no changes. Writes the report "Consent Review - Teams Apps - Report". Whether a permission is ACCEPTABLE is a security/compliance decision — this surfaces and flags; it does not approve or revoke. HOW TO RUN .\TEAMSAPPS-ConsentReport.ps1 -CsvPath C:\Temp\TEAMSAPPS-Consent.csv (PowerShell 5.1+/7+; Microsoft Graph SDK) Connect: Connect-MgGraph -Scopes Application.Read.All, Directory.Read.All, DelegatedPermissionGrant.Read.All, AppRoleAssignment.Read.All EXPECTED RESULTS Console summary of apps holding high-risk permissions, and an optional CSV with one row per (app, permission, type, risk flag). No changes are made. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Read-only; the approval decision is deferred to security/compliance. > [CmdletBinding()] param([string]$CsvPath, [string[]]$HighRiskScopes = @('Mail.Read','Mail.ReadWrite','Mail.Send','Files.Read.All','Files.ReadWrite.All', 'Sites.Read.All','Sites.ReadWrite.All','Sites.FullControl.All','Chat.Read.All','ChannelMessage.Read.All', 'Directory.Read.All','Directory.ReadWrite.All','User.Read.All','User.ReadWrite.All', 'Group.ReadWrite.All','full_access_as_app','Application.ReadWrite.All'))

How to run: .\TEAMSAPPS-ConsentReport.ps1 -CsvPath C:\Temp\TEAMSAPPS-Consent.csv (PowerShell 5.1+/7+; Microsoft Graph SDK) Connect: Connect-MgGraph -Scopes Application.Read.All, Directory.Read.All, DelegatedPermissionGrant.Read.All, AppRoleAssignment.Read.All EXPECTED RESULTS Console summary of apps holding high-risk permissions, and an optional CSV with one row per (app, permission, type, risk flag). No changes are made. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Read-only; the approval decision is deferred to security/compliance. > [CmdletBinding()] param([string]$CsvPath, [string[]]$HighRiskScopes = @('Mail.Read','Mail.ReadWrite','Mail.Send','Files.Read.All','Files.ReadWrite.All', 'Sites.Read.All','Sites.ReadWrite.All','Sites.FullControl.All','Chat.Read.All','ChannelMessage.Read.All', 'Directory.Read.All','Directory.ReadWrite.All','User.Read.All','User.ReadWrite.All', 'Group.ReadWrite.All','full_access_as_app','Application.ReadWrite.All'))

TEAMSAPPS-Discovery.ps1

Purpose: Builds the authoritative Teams app-governance baseline: every app permission policy and app setup policy (MicrosoftTeams module), the full Teams app catalog with each app's distribution method (store / organization / sideloaded) from Microsoft Graph, and the apps actually installed per team. Risk-flags sideloaded (uploaded) and organization-published (LOB) apps for review. Read-only — makes no changes. HOW TO RUN .\TEAMSAPPS-Discovery.ps1 -CsvPath C:\Temp\TEAMSAPPS-Catalog.csv (PowerShell 5.1+/7+; MicrosoftTeams module + Microsoft Graph SDK) Connect: Connect-MicrosoftTeams and Connect-MgGraph with read scopes AppCatalog.Read.All, Directory.Read.All, Team.ReadBasic.All, TeamsAppInstallation.ReadForTeam.All. EXPECTED RESULTS Console summary: count of permission/setup policies, and a catalog table grouped by distribution method with sideloaded/LOB apps flagged. Optional per-app CSV. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Read-only; no changes, no stored credentials. Some org-wide app settings are admin-center-only — verify PowerShell/Graph parity in your tenant. > [CmdletBinding()] param([string]$CsvPath, [switch]$IncludeInstalled # also enumerate installed apps per team (slower on large tenants))

How to run: .\TEAMSAPPS-Discovery.ps1 -CsvPath C:\Temp\TEAMSAPPS-Catalog.csv (PowerShell 5.1+/7+; MicrosoftTeams module + Microsoft Graph SDK) Connect: Connect-MicrosoftTeams and Connect-MgGraph with read scopes AppCatalog.Read.All, Directory.Read.All, Team.ReadBasic.All, TeamsAppInstallation.ReadForTeam.All. EXPECTED RESULTS Console summary: count of permission/setup policies, and a catalog table grouped by distribution method with sideloaded/LOB apps

flagged. Optional per-app CSV. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Read-only; no changes, no stored credentials. Some org-wide app settings are admin-center-only — verify PowerShell/Graph parity in your tenant. > [CmdletBinding()] param([string]$CsvPath, [switch]$IncludeInstalled # also enumerate installed apps per team (slower on large tenants))

TEAMSAPPS-Set-ScopeTargeting.ps1

Purpose: Scope/health helper for the rollout: assigns the app permission and app setup policies to each member of a pilot group (or an explicit UPN list) via Grant-CsTeamsAppPermissionPolicy / Grant-CsTeamsAppSetupPolicy, and can RESET a population back to the Global (Org-wide default) policies for rollback. SAFE BY DEFAULT — runs in -WhatIf mode (prints the assignments it WOULD make) unless -Commit is supplied. HOW TO RUN Dry-run (default, NO changes): .\TEAMSAPPS-Set-ScopeTargeting.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' Apply: .\TEAMSAPPS-Set-ScopeTargeting.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' -Commit Rollback a population to Global: .\TEAMSAPPS-Set-ScopeTargeting.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' -Reset -Commit (PowerShell 5.1+/7+; MicrosoftTeams module + Microsoft Graph SDK for group expansion) EXPECTED RESULTS -WhatIf: one line per user showing the assignment (or reset) that WOULD be made; nothing changes. -Commit: the policies are granted (or reset to Global) for each user; a per-user result is printed. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Default mode makes NO changes. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyGroup, [string]$PermissionPolicyName = 'App Permission Policy - Pilot - Pilot', [string]$SetupPolicyName = 'App Setup Policy - Standard - Prod', [switch]$Reset, # assign $null (Global / Org-wide default) instead — rollback [switch]$Commit # absent = -WhatIf (default, no changes))

How to run: Dry-run (default, NO changes): .\TEAMSAPPS-Set-ScopeTargeting.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' Apply: .\TEAMSAPPS-Set-ScopeTargeting.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' -Commit Rollback a population to Global: .\TEAMSAPPS-Set-ScopeTargeting.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' -Reset -Commit (PowerShell 5.1+/7+; MicrosoftTeams module + Microsoft Graph SDK for group expansion) EXPECTED RESULTS -WhatIf: one line per user showing the assignment (or reset) that WOULD be made; nothing changes. -Commit: the policies are granted (or reset to Global) for each user; a per-user result is printed. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Default mode makes NO changes. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyGroup, [string]$PermissionPolicyName = 'App Permission Policy - Pilot - Pilot', [string]$SetupPolicyName = 'App Setup Policy - Standard - Prod', [switch]$Reset, # assign $null (Global / Org-wide default) instead — rollback [switch]$Commit # absent = -WhatIf (default, no changes))

TEAMSAPPS-Validation.ps1

Purpose: Validates the governance rollout: for each user in a pilot group (or an explicit user list), confirms the effective app permission policy and app setup policy match the intended names, and summarises the permission policy's allow/block mode so you can confirm it allows only the vetted set. Read-only — makes no changes; this is the pilot pass/fail gate before org-wide expansion. HOW TO RUN .\TEAMSAPPS-Validation.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' `-ExpectedPermissionPolicy 'App Permission Policy - Pilot - Pilot'` -ExpectedSetupPolicy 'App Setup Policy - Standard - Prod' (PowerShell 5.1+/7+; MicrosoftTeams module + Microsoft Graph SDK for group expansion) Connect: Connect-MicrosoftTeams; Connect-MgGraph -Scopes GroupMember.Read.All, User.Read.All EXPECTED RESULTS A per-user PASS/FAIL table

(assigned vs expected permission and setup policy) and an overall gate result. No changes are made. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Read-only validation. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyGroup, [string]$ExpectedPermissionPolicy = 'App Permission Policy - Pilot - Pilot', [string]$ExpectedSetupPolicy = 'App Setup Policy - Standard - Prod', [string]$CsvPath)

How to run: .\TEAMSAPPS-Validation.ps1 -PolicyGroup 'App Governance Pilot - Users - Pilot' `-ExpectedPermissionPolicy 'App Permission Policy - Pilot - Pilot'` -ExpectedSetupPolicy 'App Setup Policy - Standard - Prod' (PowerShell 5.1+/7+; MicrosoftTeams module + Microsoft Graph SDK for group expansion) Connect: Connect-MicrosoftTeams; Connect-MgGraph -Scopes GroupMember.Read.All, User.Read.All EXPECTED RESULTS A per-user PASS/FAIL table (assigned vs expected permission and setup policy) and an overall gate result. No changes are made. .NOTES Teams — Microsoft Teams Apps & Integration Governance. Read-only validation. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyGroup, [string]$ExpectedPermissionPolicy = 'App Permission Policy - Pilot - Pilot', [string]$ExpectedSetupPolicy = 'App Setup Policy - Standard - Prod', [string]$CsvPath)